

B・C・D 担当者向け 開発・テスト・CI 運用マニュアル

Vite / React + Vitest + GitHub Actions 前提

この資料は、B・C・D 担当者が自分の担当機能を実装し、単体テストを書き、Pull Request を作成し、GitHub Actions の CI 結果を確認するための手順をまとめたものです。

開発初心者でも作業順序を間違えにくいように、コマンドと確認箇所を具体的に示しています。

1. この開発で使う授業技術

授業技術	この開発で行うこと
ソフトウェアテスト技法	担当機能の一部に対して単体テストを書く。主に入力チェック、並び替え、時刻計算などの関数を検査する。
継続的インテグレーション	GitHub Actions で、push 時と Pull Request 作成時に npm test と npm run build を自動実行する。

GitHub に push したり Pull Request を作成したりすると、GitHub Actions が自動でテストとビルドを実行します。CI が成功してから main ブランチへマージします。

2. 全体の作業フロー

1. main ブランチを最新化する
2. 自分用の作業ブランチを作る
3. 担当機能を実装する
4. 必要なテストを書く
5. ローカルで npm test と npm run build を実行する
6. commit する
7. GitHub に push する
8. Pull Request を作成する
9. GitHub Actions の結果を確認する
10. CI が成功したらチームに報告する

3. 絶対に守るルール

ルール	理由
main ブランチに直接 push しない	他の人の作業と衝突しやすくなり、壊れた状態が全員に共有されるため。
作業前に必ず git pull origin main する	古い状態から作業すると、コンフリクトや動作不良が起きやすいため。
1つの担当機能につき1つのブランチを作る	変更範囲を分かりやすくし、レビューしやすくするため。
Pull Request 前に npm test と npm run build を実行する	GitHub 上で CI が失敗する可能性を下げるため。
CI が赤い状態でマージしない	壊れたコードが main に入るため。
他人の担当ファイルを不用意に変更しない	コンフリクトを避けるため。

4. 初回だけ行う環境準備

4.1 Windows PowerShell

PowerShell
git clone リポジトリ URL

```
cd リポジトリ名
npm install
npm run dev
```

別ターミナルで、テストとビルドを確認します。

```
PowerShell
npm test
npm run build
```

4.2 Linux / macOS

```
bash
git clone リポジトリ URL
cd リポジトリ名
npm install
npm run dev
```

別ターミナルで、テストとビルドを確認します。

```
bash
npm test
npm run build
```

ここで npm test と npm run build が成功すれば、開発環境の準備は完了です。

5. すでにリポジトリを持っている場合

作業前に、必ず main ブランチを最新化します。

5.1 Windows PowerShell

```
PowerShell
git checkout main
git pull origin main
npm install
npm test
npm run build
```

5.2 Linux / macOS

```
bash
git checkout main
git pull origin main
npm install
npm test
npm run build
```

6. 作業ブランチを作る

各担当者は、自分の作業用ブランチを作ってから実装します。

担当者	ブランチ名の例
B 担当	feature/b-post-form
C 担当	feature/c-board-list

D 担当

feature/d-random-timer

6.1 Windows PowerShell

PowerShell

```
git checkout main
git pull origin main
git checkout -b feature/自分の担当機能名
```

6.2 Linux / macOS

bash

```
git checkout main
git pull origin main
git checkout -b feature/自分の担当機能名
```

例:

bash

```
git checkout -b feature/b-post-form
```

7. 実装するときの基本方針

画面に直接すべての処理を書くのではなく、できるだけ「画面表示」と「処理・判定」を分けます。

種類	置き場所の例	内容
画面表示	src/components	ボタン、入力欄、投稿一覧など
処理・判定	src/utils	入力チェック、並び替え、時刻計算など
テスト	src/**/*.test.js	処理が正しいか確認するコード

テストしやすいのは、src/utils に置いた関数です。次のような処理は、できるだけ関数として分離してください。

- 空文字の投稿を拒否する処理
- 投稿を新しい順に並べる処理
- ユーザー名が正しいか確認する処理
- ランダム時刻を作る処理
- 画像ファイル名や URL を確認する処理

8. テストを書く場所

テストファイルは、テスト対象のファイルと近い場所に作ります。

text

```
src/utils/postRules.js
src/utils/postRules.test.js

src/utils/timeRules.js
src/utils/timeRules.test.js
```

ファイル名は、基本的に「対象ファイル名.test.js」にします。

9. テストの最小例

投稿内容を検証する関数を作る場合の例です。

9.1 src/utils/postRules.js

JavaScript

```
export function validatePostText(text) {
  if (typeof text !== "string") {
    return { ok: false, reason: "投稿内容が文字列ではありません" };
  }

  const trimmed = text.trim();

  if (trimmed.length === 0) {
    return { ok: false, reason: "投稿内容が空です" };
  }

  if (trimmed.length > 100) {
    return { ok: false, reason: "投稿内容が長すぎます" };
  }

  return { ok: true, reason: "" };
}
```

9.2 src/utils/postRules.test.js

JavaScript

```
import { describe, expect, test } from "vitest";
import { validatePostText } from "../postRules";

describe("validatePostText", () => {
  test("空文字の投稿を拒否する", () => {
    expect(validatePostText("")).ok.toBe(false);
    expect(validatePostText(" ").ok).toBe(false);
  });

  test("通常の投稿を許可する", () => {
    expect(validatePostText("今日のスクショ").ok).toBe(true);
  });

  test("長すぎる投稿を拒否する", () => {
    const text = "a".repeat(101);
    expect(validatePostText(text).ok).toBe(false);
  });
});
```

10. 各担当者が最低限書くべきテスト

各担当者は、自分の担当機能について最低1つ、できれば2〜3個のテストを書いてください。画面全体のテストは難しいので、まずは関数単位のテストで構いません。

担当	テスト例
B 担当	入力为空のとき投稿できないことを確認する。
C 担当	投稿一覧が新しい順に並ぶことを確認する。
D 担当	ランダム時刻が 0:00〜23:59 の範囲に収まることを確認する。

11. ローカルで確認する

実装後、必ず次の2つを実行してください。

11.1 Windows PowerShell

```
PowerShell
npm test
npm run build
```

11.2 Linux / macOS

```
bash
npm test
npm run build
```

両方成功すれば、Pull Request を作成してよい状態です。

npm test の成功例:

```
text
Test Files  1 passed
Tests       3 passed
```

npm run build の成功例:

```
text
✓ built in ...
```

12. commit する

作業が終わったら、まず変更内容を確認します。

12.1 Windows PowerShell

```
PowerShell
git status
git add .
git commit -m "担当機能の実装とテストを追加"
```

12.2 Linux / macOS

```
bash
git status
git add .
git commit -m "担当機能の実装とテストを追加"
```

commit メッセージは、何をしたか分かる内容にしてください。

良い例	悪い例
add post validation tests	update
add board sorting function	fix
add random notification time logic	aaa
fix post form validation	作業

13. GitHub に push する

最初の push では、次のように実行します。

13.1 Windows PowerShell

PowerShell

```
git push -u origin ブランチ名
```

13.2 Linux / macOS

bash

```
git push -u origin ブランチ名
```

例:

bash

```
git push -u origin feature/b-post-form
```

2 回目以降は、基本的に git push で構いません。

14. Pull Request を作る

GitHub でリポジトリを開くと、push したブランチから Pull Request を作成できます。Pull Request の向きは次のようにしてください。

text

base: main

compare: 自分の作業ブランチ

14.1 PR タイトルの例

text

B 担当: 投稿フォームの入力チェックを追加

C 担当: 投稿一覧の表示処理を追加

D 担当: ランダム通知時刻の生成処理を追加

14.2 PR 本文テンプレート

Markdown

担当者

B / C / D のいずれかを書く

実装内容

-
-

追加・変更したファイル

-
-

テスト内容

- [] npm test を実行した
- [] npm run build を実行した
- [] 担当機能の動作を画面で確認した

備考

特になければ「なし」

15. GitHub Actions の確認方法

11. GitHub で Pull Request を開く
12. 下の方にある Checks を確認する
13. CI が緑色になっているか確認する

成功している状態の例:

```
text
✓ CI / test-and-build
```

失敗している状態の例:

```
text
✗ CI / test-and-build
```

CI が失敗している場合は、main にマージしないでください。

16. CI が失敗したときの直し方

14. Pull Request の Checks を開く
15. 失敗している Job をクリックする
16. 赤くなっているステップを確認する
17. エラーメッセージを読む
18. 自分の PC で同じコマンドを実行する
19. 修正する
20. commit して push する
21. CI が再実行されるので再確認する

ローカルで実行するコマンド:

```
bash
npm test
npm run build
```

17. よくあるエラーと原因

17.1 npm test が失敗する

原因の例:

- 期待値と実際の値が違う
- テスト対象の関数名を間違えている
- import のパスを間違えている
- テスト対象の関数を export していない

確認する場所:

```
text
src/**/*.test.js
src/utils/*.js
```

17.2 npm run build が失敗する

原因の例:

- 存在しないファイルを import している
- ファイル名の大文字・小文字が一致していない
- 使っていない変数や構文エラーがある
- React コンポーネントの書き方に誤りがある

特に注意する点: Windows では動くが、GitHub Actions の Linux 環境では失敗することがあります。ファイル名の大文字・小文字は必ず一致させてください。

JavaScript

```
// 例
import PostCard from "./PostCard";

// 実際のファイル名も PostCard.jsx にする。
// postCard.jsx や Postcard.jsx だと CI で失敗する可能性がある。
```

17.3 Missing script: "test" と出る

原因: package.json に test スクリプトがない、または main ブランチが最新ではない可能性があります。

bash

```
git checkout main
git pull origin main
git checkout 自分のブランチ名
```

17.4 import でエラが出る

例:

text

```
Cannot find module "./postRules"
```

確認すること:

- ファイル名が合っているか
- 拡張子の扱いが正しいか
- 相対パスが正しいか
- export / import の名前が一致しているか

JavaScript

```
export function validatePostText(text) {
  // ...
}

import { validatePostText } from "./postRules";
```

18. コンフリクトした場合

Pull Request でコンフリクトが出た場合は、自分だけで無理に解消しないでください。まず、次を共有してください。

- 自分のブランチ名

- コンフリクトしているファイル名
- GitHub 上の Pull Request URL
- どの部分で困っているか

コンフリクトは、複数人が同じファイルの同じ場所を変更したときに起きやすいです。

特にコンフリクトしやすいファイル:

```
text
src/App.jsx
package.json
package-lock.json
```

これらを変更する場合は、他の担当者と事前に確認してください。

19. Pull Request 前チェックリスト

Markdown

- ☐ main ブランチを最新化してから作業した
- ☐ 自分用の作業ブランチで作業した
- ☐ 担当機能が画面上で動く
- ☐ テストを最低 1 つ追加した
- ☐ npm test が成功した
- ☐ npm run build が成功した
- ☐ 不要な console.log を残していない
- ☐ 他人の担当ファイルを不要に変更していない
- ☐ Pull Request の説明を書いた
- ☐ GitHub Actions の CI 結果を確認した

20. B・C・D 担当者の成果物

各担当者は、最終的に次の 4 つを残してください。

22. 実装したコード

23. 追加したテストコード

24. Pull Request

25. GitHub Actions の成功結果

発表時には、次のように説明できます。

text

各担当者は、自分の担当機能をブランチ上で実装し、関数単位のテストを追加した。
Pull Request 作成時には GitHub Actions により npm test と npm run build が自動実行され、
テスト失敗やビルド失敗が main ブランチに入る前に検出できるようにした。

21. 作業完了時の連絡テンプレート

Markdown

担当: B / C / D

作業ブランチ:
feature/...

Pull Request:
URL を書く

実装内容:

-
-

確認:

- [] npm test 成功
- [] npm run build 成功
- [] CI 成功

困っていること:

なし / 内容を書く

22. 最低限これだけであればよい

- 担当機能が動く
- テストが1つ以上ある
- npm test が通る
- npm run build が通る
- Pull Request を作れる
- GitHub Actions が緑になる

この状態になれば、授業で扱った「ソフトウェアテスト技法」と「継続的インテグレーション」を開発に使ったと言えます。